

UNIVERSIDAD POLITÉCNICA TERRITORIAL DEL OESTE DE SUCRE "CLODOSBALDO RUSSIÁN"

Maestría en Informática Aplicada, Mención Desarrollo de Software

TRABAJO INTEGRADOR

Tópicos Avanzados en Base de Datos

Aplicación de XML, Bases de Datos Activas/Deductivas y Data Warehouse en el Sistema de Gestión Penitenciaria (SGP)

Autores:

Henry García C.I. 997869

Asdrúbal Hernández C.I. 17445063

Yusvania Jiménez C.I. 15740626

Franklin Mundaray C.I. 12660350

Docente: Prof. Ana Teresa Fuentes M.

Fecha de Entrega: 27 de Octubre de 2026

Índice general

1. Bases de Datos XML Aplicadas al SGP **2**
- 1.1. Introducción . . . 2
- 1.2. Marco Teórico . . . 2
  - 1.2.1. XML . . . 2
  - 1.2.2. Ventajas de Implementar XML en Bases de Datos . . . 2
  - 1.2.3. Desventajas y Limitaciones de XML . . . 3
  - 1.2.4. Diferencias Clave: Relacional vs XML . . . 4
- 1.3. Diseño e Implementación Práctica . . . 4
  - 1.3.1. Identificación del Caso de Uso . . . 4
  - 1.3.2. Construcción de la consulta y generación del documento XML en PostgreSQL . . . 4
  - 1.3.3. Validación mediante Esquema XSD . . . 5
- 1.4. Análisis de resultados . . . 6

2. Bases de Datos Activas y Deductivas **7**
- 2.1. Propósito . . . 7
- 2.2. Investigación Documental . . . 7
  - 2.2.1. Bases de Datos Activas . . . 7
  - 2.2.2. Bases de Datos Deductivas y Herramientas Especializadas . . . 7
- 2.3. Implementación Práctica en PostgreSQL . . . 8
  - 2.3.1. Regla Activa 1: Control de Capacidad de Celdas . . . 8
  - 2.3.2. Regla Activa 2: Bloqueo de Redención por Falta Grave . . . 8
  - 2.3.3. Mecanismo de Inferencia (Deductivo) mediante SQL . . . 9

3. Diseño de un Data Warehouse **11**
- 3.1. Investigación y Necesidades Analíticas . . . 11
- 3.2. Granularidad y Modelo Dimensional . . . 11
- 3.3. Diccionario de Datos . . . 12
- 3.4. Indicadores y Medidas . . . 12
- 3.5. Proceso ETL (Extract, Transform, Load) . . . 12

4. Conclusiones y Recomendaciones **13**

5. Anexos **14**

Capítulo 1

Bases de Datos XML Aplicadas al SGP

1.1. Introducción

Este informe permite investigar el uso de bases de datos orientadas a documentos XML, sus características de representación e intercambio, y desarrollar un caso de uso aplicado que permita estructurar, consultar y validar información crítica del Sistema de Gestión Penitenciaria (SGP), demostrando la flexibilidad de las tecnologías documentales frente a los modelos relacionales tradicionales.

El lenguaje de marcado extensible (XML) permite la estructuración jerárquica de datos, facilitando el intercambio de información heterogénea entre sistemas informáticos independientes.

1.2. Marco Teórico

1.2.1. XML

El lenguaje XML (Extensible Markup Language) es un estándar del W3C diseñado para almacenar y transportar datos de manera estructurada y legible tanto para humanos como para máquinas. A diferencia de los formatos planos, XML utiliza etiquetas personalizables para definir una jerarquía en forma de árbol (nodos, elementos y atributos).

En el contexto de los sistemas de gestión de bases de datos (SGBD), la evolución tecnológica dio paso a las bases de datos XML (y al soporte de tipos de datos XML nativos en motores relacionales como PostgreSQL). Esto permite almacenar documentos XML completos, indexarlos y consultarlos utilizando lenguajes estándar de consulta como XQuery y XPath, combinando la flexibilidad documental con la potencia de transacciones robustas (ACID).

1.2.2. Ventajas de Implementar XML en Bases de Datos

- **Flexibilidad y esquemas semiestructurados:** Permite almacenar datos cuyos atributos varían entre registros sin necesidad de modificar constante-

mente la estructura de la tabla.

- **Representación de jerarquías complejas:** Es ideal para modelar datos con relaciones anidadas complejas en una sola estructura de árbol natural.
- **Intercambio nativo de datos:** Facilita la integración con servicios web (SOAP, APIs) y aplicaciones externas.
- **Autodescripción:** Cada elemento contiene metadatos sobre sí mismo (las etiquetas), lo que hace que los datos sean legibles sin depender estrictamente de un diccionario externo.

1.2.3. Desventajas y Limitaciones de XML

- **Alto costo de almacenamiento:** El uso de etiquetas repetitivas incrementa notablemente el tamaño físico de la base de datos.
- **Rendimiento:** Las operaciones de lectura y actualización a nivel granular suelen ser más lentas porque requieren analizar (parsear) el árbol XML completo.
- **Complejidad en la indexación:** Configurar índices eficientes para consultas XML requiere mayor consumo de recursos de CPU y memoria.
- **Curva de aprendizaje:** Requiere dominar lenguajes adicionales como XPath y XQuery.

1.2.4. Diferencias Clave: Relacional vs XML

Característica	Bases de Datos Relacionales	Bases de Datos / Campos XML
Modelo de Datos	Tablas con filas y columnas (esquema rígido).	Árboles jerárquicos y documentos semiestructurados.
Evolución del Esquema	Requiere comandos pesados (`ALTER TABLE`).	Altamente flexible; se añaden nuevos elementos sin alterar la estructura.
Normalización	Requiere normalización para evitar redundancias.	Permite la desnormalización natural (documento autocontenido).
Lenguaje de Consulta	SQL, optimizado para operaciones matemáticas y `JOIN`.	XPath y XQuery, diseñados para navegar por rutas jerárquicas.
Eficiencia de Espacio	Altamente eficiente; tipos de datos primitivos.	Menos eficiente debido al almacenamiento repetitivo de etiquetas.
Transaccionalidad	Excelente soporte maduro (ACID).	Soportado en SGBD híbridos, pero más costoso a nivel de subelementos.

Cuadro 1.1: Comparativa entre bases de datos relacionales y XML.

1.3. Diseño e Implementación Práctica

1.3.1. Identificación del Caso de Uso

Para el SGP, se seleccionó la **Ficha o Registro Digital Individual del Recluso**, enfocada en empaquetar los datos demográficos y operativos (cédula, nombres, apellidos, nivel de riesgo, estado operativo, sexo y fecha de nacimiento). La justificación radica en que este formato simplifica la exportación e interoperabilidad de perfiles individuales hacia entes ministeriales o judiciales externos sin comprometer el esquema transaccional relacional central.

1.3.2. Construcción de la consulta y generación del documento XML en PostgreSQL

Mediante el uso de funciones nativas (`XMLELEMENT`, `XMLATTRIBUTES` y `XMLFOREST`), se transformó una fila normalizada de la tabla `reclusos` en una estructura documental jerárquica.

```
SELECT XMLELEMENT(NAME "expediente",
    XMLELEMENT(NAME "recluso",
        XMLFOREST(r.cedula_identidad AS "cedula",
                  r.nombres || ' ' || r.apellidos AS "
   nombre_completo",
                  r.estado_operativo)
```

```
    ),
    XMLELEMENT(NAME "causas",
        XMLAGG(
            XMLELEMENT(NAME "causa",
                XMLFOREST(c.numero_unico, c.delito_principal AS "delito",
                          c.estado_procesal, c.dias_condena)
            )
        )
    )
) AS expediente_xml
FROM reclusos r
LEFT JOIN recluso_causa rc ON r.id = rc.recluso_id
LEFT JOIN causas_penales c ON rc.causa_penal_id = c.id
WHERE r.id = 7
GROUP BY r.id, r.cedula_identidad, r.nombres, r.apellidos, r.estado_operativo;
```

Listing 1.1: Generación de XML del Expediente Único en PostgreSQL

Como se evidencia en el **Anexo 1**, la ejecución en consola devuelve el documento XML completo con las causas asociadas a la reclusa, demostrando la capacidad de PostgreSQL para generar documentos válidos en tiempo real.

1.3.3. Validación mediante Esquema XSD

Para garantizar la integridad y calidad del documento generado, se diseñó un esquema de validación (`expediente.xsd`), el cual establece restricciones estrictas sobre los tipos de datos y los valores permitidos (como el control estricto del campo sexo a los caracteres 'M' o 'F').

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="recluso">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="cedula" type="xs:string"/>
        <xs:element name="nombres" type="xs:string"/>
        <xs:element name="apellidos" type="xs:string"/>
        <xs:element name="nivel_riesgo" type="xs:string"/>
        <xs:element name="estado_operativo" type="xs:string"/>
        <xs:element name="sexo" type="xs:string"/>
        <xs:element name="fecha_nacimiento" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="id_sistema" type="xs:integer" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Listing 1.2: Fragmento del Esquema XSD de Validación

1.4. Análisis de resultados

La implementación demostró que con PostgreSQL podemos tender un puente sumamente eficiente entre la persistencia relacional transaccional y la gestión documental orientada a XML. El uso de validadores externos basados en XSD confirmó que los documentos generados cumplen de forma estricta con las especificaciones técnicas requeridas.

Capítulo 2

Bases de Datos Activas y Deductivas

2.1. Propósito

Comprender los fundamentos teóricos y prácticos de las bases de datos activas y deductivas, analizando los mecanismos de eventos, condiciones y acciones (ECA), así como la inferencia lógica de conocimiento. La implementación práctica se llevó a cabo en el motor relacional PostgreSQL utilizado en el SGP.

2.2. Investigación Documental

2.2.1. Bases de Datos Activas

Extienden el modelo relacional tradicional incorporando el paradigma de reacción automática ante sucesos específicos mediante el uso de reglas de tipo Evento-Condición-Acción (ECA).

- **Eventos:** Operaciones de manipulación de datos (INSERT, UPDATE, DELETE).
- **Condiciones:** Expresiones lógicas evaluadas al ocurrir el evento.
- **Acciones:** Procedimientos ejecutados por el motor (triggers) que modifican el estado o lanzan excepciones de integridad.

Consideraciones: Su uso excesivo puede generar efectos secundarios, bucles de ejecución recursiva infinita y sobrecarga en el procesamiento transaccional.

2.2.2. Bases de Datos Deductivas y Herramientas Especializadas

Integran la teoría de bases de datos con la lógica matemática. Permiten definir hechos almacenados y reglas de inferencia para derivar lógicamente nuevos conocimientos que no se encuentran explícitamente guardados en las tablas base.

- **SWI-Prolog:** Entorno de programación lógica basado en la lógica de predicados de primer orden. Potente para razonamiento simbólico, pero con limitaciones de rendimiento en grandes volúmenes de datos masivos.
- **Datalog:** Lenguaje declarativo diseñado para bases de datos deductivas. Garantiza la terminación de las inferencias y es adecuado para análisis de grafos y normativas complejas.
- **Escenario de elección en el SGP:** Resolver validaciones masivas de celdas o transacciones concurrentes mediante SQL y Triggers nativos resulta drásticamente más eficiente que delegar la persistencia transaccional a motores lógicos puros.

2.3. Implementación Práctica en PostgreSQL

2.3.1. Regla Activa 1: Control de Capacidad de Celdas

Para garantizar la integridad operativa del SGP y evitar la sobrepoblación, se diseñó una regla activa de tipo `BEFORE INSERT OR UPDATE` sobre la tabla `reclusos`, la cual evalúa automáticamente la capacidad máxima configurada en la tabla `celdas`.

```
CREATE OR REPLACE FUNCTION public.fn_verificar_capacidad_celda()
RETURNS TRIGGER AS $$
DECLARE
    v_capacidad_maxima INTEGER;
    v_total_actual INTEGER;
BEGIN
    IF NEW.celda_id IS NULL THEN RETURN NEW; END IF;
    SELECT capacidad_maxima INTO v_capacidad_maxima FROM public.
    celdas WHERE id = NEW.celda_id;
    SELECT COUNT(*) INTO v_total_actual FROM public.reclusos WHERE
    celda_id = NEW.celda_id AND id <> COALESCE(NEW.id, 0);
    IF v_total_actual >= v_capacidad_maxima THEN
        RAISE EXCEPTION 'ERROR DE INTEGRIDAD: La celda seleccionada
    ha alcanzado su capacidad m xima permitida (%) y no admite
    m s reclusos.', v_capacidad_maxima;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_control_capacidad_celda BEFORE INSERT OR UPDATE
    OF celda_id ON public.reclusos FOR EACH ROW EXECUTE FUNCTION
    public.fn_verificar_capacidad_celda();
```

Listing 2.1: Trigger de control de capacidad en celdas

2.3.2. Regla Activa 2: Bloqueo de Redención por Falta Grave

En el SGP, un recluso no puede redimir pena si tiene una falta disciplinaria grave activa. Se implementó un trigger que impide la transacción.

```sql
CREATE OR REPLACE FUNCTION validar_redencion_por_falta()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM faltas_disciplinarias
        WHERE recluso_id = NEW.recluso_id
          AND gravedad = 'Grave'
          AND activa = true
    ) THEN
        RAISE EXCEPTION 'El recluso tiene una falta disciplinaria grave activa. No puede redimir pena.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_validar_redencion
BEFORE INSERT OR UPDATE ON redenciones
FOR EACH ROW EXECUTE FUNCTION validar_redencion_por_falta();
```

Listing 2.2: Trigger para bloquear redención por falta grave

Evidencias del Comportamiento: En el **Anexo 2** se confirma la creación del trigger. En el **Anexo 3**, al intentar registrar una redención para una reclusa con falta grave activa, el motor abortó la transacción informando la infracción, demostrando la efectividad de la regla.

2.3.3. Mecanismo de Inferencia (Deductivo) mediante SQL

Para demostrar la capacidad deductiva, se diseñó una consulta con Expresiones de Tabla Comunes (CTE) que infiere de manera automática el estatus de elegibilidad para beneficios de redención combinando relaciones dispersas.

```sql
WITH ResumenPenal AS (
    SELECT
        r.id AS recluso_id,
        r.cedula_identidad,
        CONCAT(r.nombres, ' ', r.apellidos) AS nombre_completo,
        r.estado_operativo,
        COALESCE(SUM(cp.dias_condena), 0) AS total_dias_condena,
        COALESCE(SUM(rd.dias_redimidos), 0) AS total_dias_redimidos
    FROM public.reclusos r
    LEFT JOIN public.recluso_causa rc ON rc.recluso_id = r.id
    LEFT JOIN public.causas_penales cp ON rc.causa_penal_id = cp.id
    LEFT JOIN public.redenciones rd ON rd.recluso_id = r.id AND rd.avalado = true
    WHERE r.estado_operativo = 'Activo'
    GROUP BY r.id, r.cedula_identidad, r.nombres, r.apellidos, r.estado_operativo
)
SELECT
    recluso_id,
    cedula_identidad,
    nombre_completo,
    total_dias_condena,
```

```
    total_dias_redimidos,
    CASE
        WHEN total_dias_condena > 0 AND (total_dias_redimidos::
   NUMERIC / total_dias_condena::NUMERIC) >= 0.10
        THEN 'ELEGIBLE PARA BENEFICIO (>=10% Redimido)'
        WHEN total_dias_condena = 0
        THEN 'PENDIENTE DE SENTENCIA FIRME'
        ELSE 'EN PROCESO DE CUMPLIMIENTO'
    END AS estatus_inferido_beneficio
FROM ResumenPenal;
```

Listing 2.3: Inferencia de Elegibilidad para Beneficios de Redención

Capítulo 3

Diseño de un Data Warehouse

3.1. Investigación y Necesidades Analíticas

Un Data Warehouse (DW) integra datos de múltiples fuentes (OLTP) para facilitar la toma de decisiones. Utilizaremos un Esquema Estrella por su simplicidad y velocidad.

Preguntas de Negocio para el SGP:

1. ¿Cuál es la tasa de ocupación promedio por centro penitenciario en el último año?
2. ¿Cómo ha evolucionado la cantidad de redenciones de pena según el tipo de actividad?
3. ¿Cuál es el tiempo promedio que pasa un recluso en el sistema antes de obtener una boleta de excarcelación?
4. ¿Qué centros penales presentan la mayor tasa de reincidencia?
5. ¿Cómo afecta la cantidad de personal de seguridad a la tasa de incidencias?

3.2. Granularidad y Modelo Dimensional

Granularidad:

- `FactPoblacionDiaria`: Una fila por cada centro penitenciario y cada día (foto del estado al final del día).
- `FactMovimiento`: Una fila por cada evento de movimiento (Ingreso, Traslado, Libertad).

3.3. Diccionario de Datos

Tabla	Atributo	Tipo	Función
FactPoblacionDiaria	`tiempo_id`	INT (FK)	Fecha de corte
	`centro_id`	INT (FK)	Centro evaluado
	`total_reclusos`	INT	Medida: Conteo de reclusos
	`tasa_ocupacion`	DECIMAL	Medida: (total/capacidad)*100
DimCentroPenal	`centro_id`	INT (PK)	Surrogate Key
	`nombre`	VARCHAR	Nombre del centro
	`nivel_seguridad`	VARCHAR	Mínima, Media, Máxima

Cuadro 3.1: Diccionario de Datos del Data Warehouse.

3.4. Indicadores y Medidas

- **Tasa de Ocupación (%):** `(Total Reclusos / Capacidad Total) * 100`. Sirve para medir el hacinamiento.
- **Índice de Redención:** `Total Días Redimidos / Total Días de Condena`. Mide la efectividad de los programas.
- **Tasa de Excarcelación:** `Cantidad de Boletas Emitidas / Población Total`. Mide el flujo de salida.

3.5. Proceso ETL (Extract, Transform, Load)

El flujo para alimentar el Data Warehouse desde el SGP (Laravel/PostgreSQL) es el siguiente:

1. **Extracción:** Un proceso programado (Cron Job) extrae datos del día anterior desde las tablas operativas.
2. **Limpieza:** Se tratan valores nulos (ej. reclusos sin celda) y se anonimizan datos personales.
3. **Transformación:** Se calculan las claves subrogadas para las dimensiones.
4. **Carga:** Se insertan los registros en `FactPoblacionDiaria` y `FactMovimiento`.
5. **Validación:** Se corren scripts para asegurar que no se dupliquen registros por fecha y centro.

Capítulo 4

Conclusiones y Recomendaciones

La integración de XML permite al SGP interoperar de forma segura con el sistema judicial, generando expedientes estructurados sin comprometer la base de datos transaccional. Las bases de datos activas (triggers de capacidad y de faltas) y deductivas (vistas y CTEs) garantizan que las reglas de negocio se cumplan a nivel del motor de base de datos, evitando errores humanos desde la aplicación. Finalmente, el Data Warehouse propuesto permitirá a la dirección del sistema penitenciario tomar decisiones basadas en datos históricos, evaluando el hacinamiento y la efectividad de las políticas de redención, cumpliendo siempre con la protección de datos sensibles.

Capítulo 5

Anexos

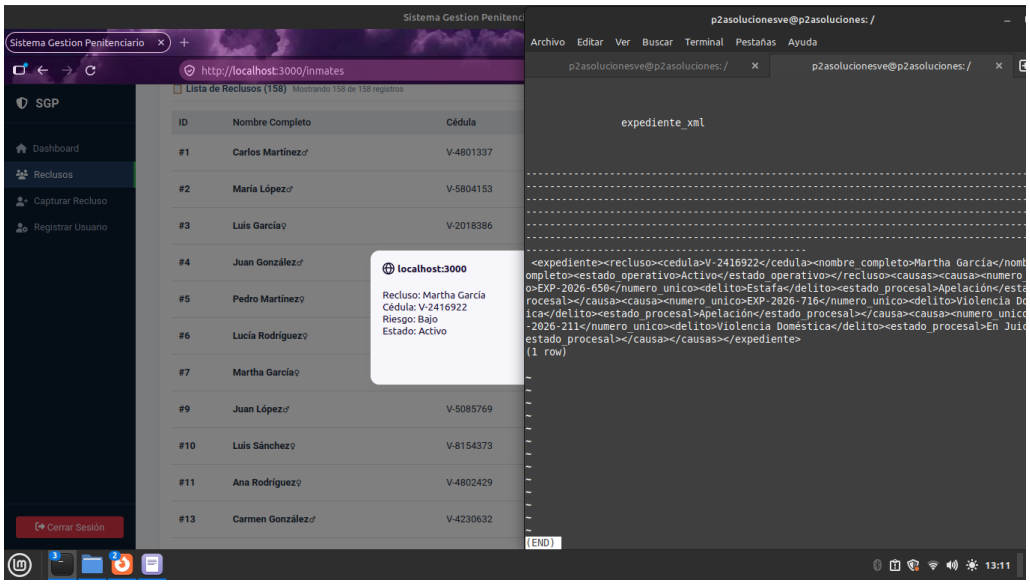


Figura 5.1: Anexo 1: Ejecución de la consulta XML en PostgreSQL y visualización en el Frontend del SGP.

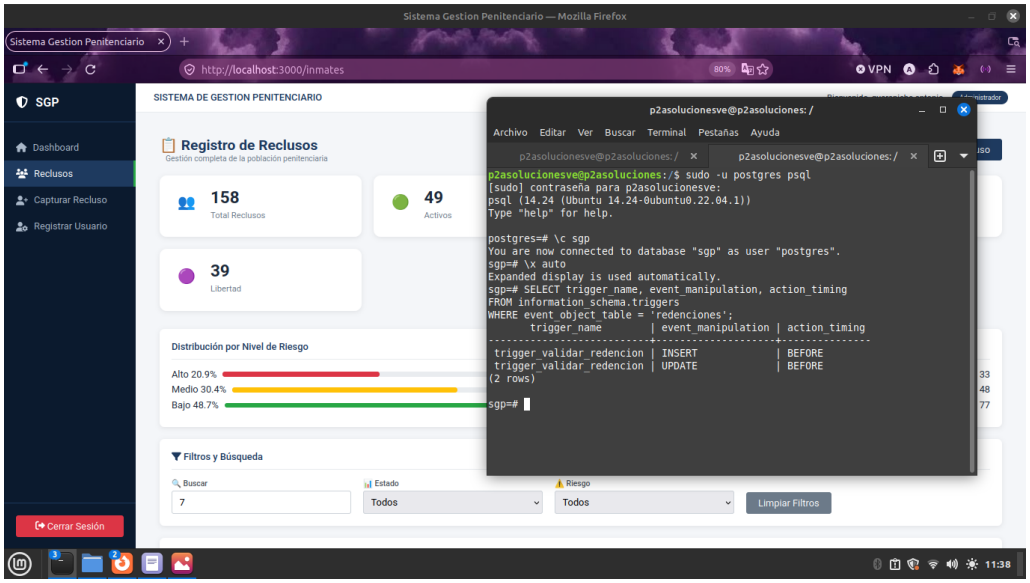


Figura 5.2: Anexo 2: Verificación de la existencia del trigger `trigger_validar_redencion` en la base de datos.

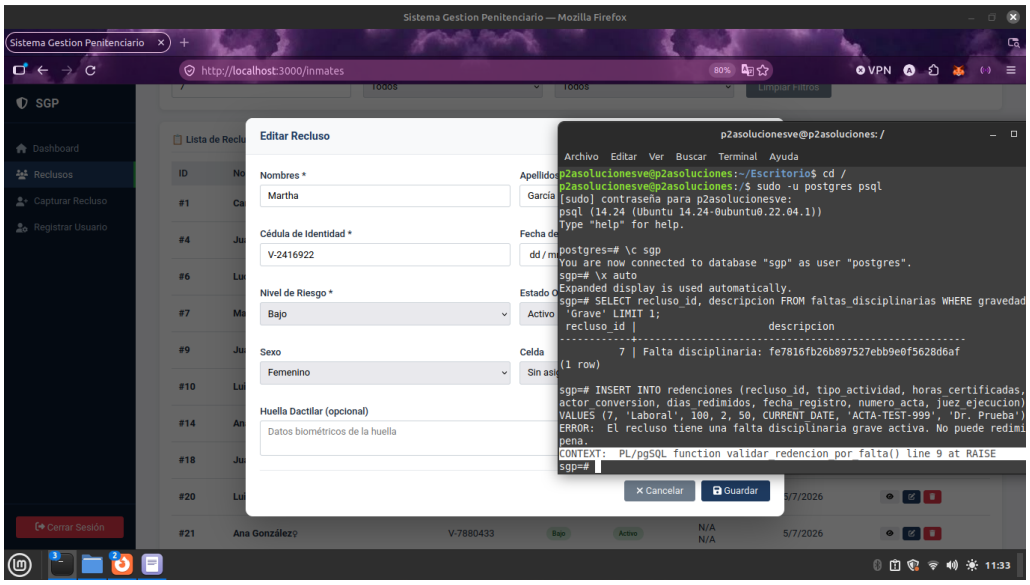


Figura 5.3: Anexo 3: Prueba de ejecución. El trigger bloquea la inserción de una redención debido a una falta grave activa.